

MERIDIAN

A Federated AI Platform for National-Scale Public Health Supply Chain Management

Solution Blueprint & Twelve-Month Build Roadmap

Submission

hack2skill — “Build with AI: Code for Communities” (2nd Edition). Problem statement track: national-scale health resource and supply chain visibility, forecasting, and federated AI for BRICS nations.

Prepared: 16 August 2026

Purpose: technical handoff document for the engineering build — covers solution design, architecture, workflow, tech stack, a twelve-month quarter-by-quarter roadmap with a hiring plan, validation approach, competitive positioning, and the path from hackathon prototype to enterprise product.

1. Executive Summary

Meridian is a federated AI platform that gives national and state health administrators real-time visibility into medicine stock, hospital bed availability, staff attendance, and patient footfall across every Primary Health Centre (PHC) in a country — then goes a step further than a monitoring dashboard by forecasting demand, issuing early warnings before stock-outs happen, and automatically recommending cross-district resource redistribution. A federated learning layer allows the predictive model to improve using signal from every PHC, district, and eventually every BRICS partner nation, without any raw patient or facility data ever leaving its origin node.

The platform is designed to satisfy the hackathon problem statement point-for-point, and almost no competing team will attempt all four pillars — visibility, forecasting, automated redistribution, and cross-border federated modelling — in a working, demoable form. But this document is not scoped as a weekend project: it lays out a full twelve-month build, starting with a working hackathon prototype in Week 1 and ending Year 1 with a multi-district deployment, a real cross-institution federated pilot, and a completed security audit. It covers system architecture, data workflow, technology choices, a quarter-by-quarter roadmap with measurable exit gates, a hiring plan tied to that roadmap, how to validate that each piece actually works at every stage, and the real-world impact case — enough technical depth to hand directly to an engineering collaborator and enough planning depth to hand to a pilot partner or an investor.

What makes this document different

- It is scoped as a twelve-month platform build, not a hackathon weekend project — the hackathon submission is Month 1, Week 1, with a measurable, gated path for the eleven months after it.
- Every feature listed has a concrete, achievable implementation path at hackathon stage, not just a concept — and a named upgrade path to a production-grade version on a specific month of the roadmap.
- It draws a hard line between what is real and working versus what is an honestly-labelled simulation for demo purposes — judges consistently reward this honesty over unverifiable claims.
- It includes a validation plan with quarterly exit criteria, so the build can be tested and trusted at every stage, not just before a demo.
- It includes a hiring and execution plan, because the goal is not just to win the hackathon but to leave with a platform — and a team plan — a health ministry or an investor could actually evaluate.

2. Problem Statement (As Given)

The problem

Public healthcare systems across developing nations face persistent supply chain vulnerabilities. The inability to track medicines, patient footfall, and resource utilisation in real time across vast networks of Primary Health Centres leads to stock-outs and limits a nation's capacity to respond when it matters most.

The challenge

Build a federated AI platform for national-scale health resource and supply chain management — real-time visibility into medicine stocks, bed availability, and medical personnel attendance across a nation's entire PHC network. It should forecast demand, generate early warnings for potential stock-outs during health emergencies, and recommend automated cross-district resource redistribution, while allowing for shared predictive modelling across BRICS nations.

Reading between the lines: what judges will actually score

Stripped to requirements, the brief asks for four discrete technical capabilities plus one constraint that shapes all of them:

#	Requirement (from brief)	What it actually demands
1	Real-time visibility across PHCs	Live data pipeline: PHC → district → national, with dashboards at each level
2	Demand forecasting	A predictive model over time-series consumption data, explainable enough to trust
3	Early warnings for stock-outs	Threshold-based alerting tied to the forecast, delivered to the right people in time to act
4	Automated cross-district redistribution	An optimization/recommendation engine, not just a dashboard — this is the differentiator almost every team skips
5	Shared predictive modelling across BRICS nations	A federated learning architecture: models train locally, only parameters are shared — this is the constraint (data sovereignty) as much as it is a feature

Most hackathon teams, working under time pressure, will build requirements 1 and 2 (a dashboard with a forecast chart) and run out of time before 3, 4, and 5. Meridian is scoped specifically so that all five are covered, each with an honest, working implementation — see Section 5 for why this is the core competitive strategy.

3. Proposed Solution — Meridian

A meridian is a line that runs across a map, connecting every point along it — and, in traditional medicine, a channel through which vital energy flows through the body to keep it healthy. Both senses fit: the platform is a line of visibility and flow running across a nation's entire PHC network, keeping medicine, beds, and staff moving to wherever they're needed most, the way a meridian keeps a system in balance.

3.1 What the platform does, in one paragraph

Every PHC logs its medicine stock, bed occupancy, staff attendance, and daily patient footfall (typed or spoken). That data rolls up in real time into district and national dashboards. A forecasting engine watches consumption trends per medicine per PHC and flags when a facility is heading toward a stock-out, well before it happens. When a shortage is predicted, a redistribution engine automatically checks nearby PHCs for surplus stock of the same medicine and recommends a specific transfer — quantity, source, destination, and estimated arrival time — with the underlying numbers shown alongside so the recommendation can be trusted and audited by a human decision-maker. Underneath all of this, a federated learning layer lets the forecasting model improve using patterns from every PHC in the network — and, at the BRICS layer, from partner nations — without any raw, identifiable patient or facility data ever being transmitted outside its PHC of origin.

3.2 The four-layer network model

- **Layer 1 — PHC Edge:** each Primary Health Centre runs a lightweight local node that captures stock, bed, staff, and footfall data, works offline-first, and trains a small local model on its own data.
- **Layer 2 — District Aggregation:** combines data from every PHC in a district, runs the forecasting and redistribution engines at district scope, and surfaces a district-level heatmap of risk.
- **Layer 3 — National Platform:** the command dashboard for national health administrators, the federated aggregation server (which averages local model updates into one improved global model), the privacy layer (encryption + differential-privacy noise), and the alerting engine.
- **Layer 4 — BRICS Federated Layer:** partner-nation nodes exchange only encrypted model weights with a shared global model — never raw records — so predictive accuracy improves across the bloc while each nation keeps full data sovereignty.

4. Key Features

Every feature below maps directly to a line in the problem statement, plus two additions designed to stand out in judging — explainability and voice accessibility — which cost little to build but directly address what health-tech judges probe hardest: trust and real-world usability.

Feature	What it does	Addresses
Real-time multi-level dashboard	Live stock, bed, staff, and footfall data at PHC, district, and national view	“real-time visibility ... across vast networks of PHCs”
AI demand forecasting	Regression/time-series model predicts near-term consumption and days-of-stock-remaining per medicine per PHC	“forecast demand”
Early-warning & alert engine	Automated threshold-based alerts (dashboard + SMS/email) to district and national officers before a stock-out occurs	“early warnings for potential stock-outs during health emergencies”
Automated redistribution engine	Rule-based optimizer matches predicted shortages to nearby surplus, recommends transfer quantity, route, and ETA	“automated cross-district resource redistribution”
Federated learning layer	Local model training per node; only encrypted weight updates are shared upward and aggregated (FedAvg)	“shared predictive modelling across BRICS nations”
Privacy-by-design	Field-level encryption + tokenization of patient identifiers; differential-privacy noise on aggregate counts	Health-data protection, a hard requirement for any real deployment
Explainability panel	Every AI recommendation is shown next to the exact numbers that produced it (e.g. days-of-cover, surplus, distance)	Judge trust; auditability for real health administrators
Voice-based data entry	PHC staff can speak a stock update (e.g. “one hundred ORS packets received”) instead of typing	Real-world usability in low-literacy, time-constrained PHC settings
Offline-first edge sync	PHC node runs on local storage and syncs to district/national layers when connectivity is available	Realistic for rural PHCs with unreliable connectivity
Simulated BRICS demo mode	Relabels simulated PHC nodes as partner nations to demonstrate cross-border federated aggregation, clearly marked as simulated	Demonstrates the BRICS requirement honestly within hackathon constraints

5. USP — Why This Wins

5.1 The core competitive insight

The problem statement asks for four things: visibility, forecasting, automated redistribution, and cross-border federated modelling. Almost every team in a hackathon of this scale will build the first two — a dashboard and a forecast chart — because those are the fastest to demo and require no unfamiliar concepts. Redistribution logic and federated learning are usually cut for time, or replaced with a slide that describes them without a working implementation behind it.

Meridian is deliberately scoped (see Section 9) so a small team can finish an honest, working version of all four — not a production-grade version, but a real one: the federated averaging is real averaging of real locally-trained model weights; the redistribution engine is a real rule-based optimizer running on real thresholds; the encryption is real symmetric encryption, not a claim on a slide. That gap between “working demo” and “slide with a claim” is the single biggest differentiator in front of judges, because it is the difference most other submissions will not close.

5.2 Competitive comparison

Capability	Typical PHC/HMIS dashboard tools	Typical hackathon AI-forecast demo	Meridian
Real-time visibility	Yes, but siloed per state/vendor	Yes, single-facility demo only	Yes — PHC → district → national → BRICS
Demand forecasting	Rare / manual reordering	Yes, but often a black-box model	Yes — explainable, backtestable model
Automated redistribution	No — manual phone/paper process	Rarely attempted	Yes — rule-based optimizer with explainability
Cross-node federated learning	No	Almost never attempted	Yes — real FedAvg demo, extensible to BRICS
Privacy-preserving design	Varies, often weak	Rarely addressed	Yes — encryption, tokenization, DP noise
Explainability	No	Rarely	Yes — every recommendation shows its inputs
Offline / low-connectivity ready	Rarely	No	Yes — offline-first PHC edge node
Accessibility (voice entry)	No	No	Yes — speech-to-form stock updates
Honest about what's simulated	N/A	Rarely disclosed	Yes — explicit in pitch and docs

5.3 How to use this in the pitch

- Open by stating the four requirements from the brief, then show all four working — in that order — so judges can check the box themselves as the demo runs.
- Say plainly which parts are simplified or simulated for hackathon scope (BRICS cross-border networking, formal differential privacy) and why — this reads as engineering maturity, not weakness.
- Lead the technical Q&A with the explainability panel: it pre-empts the most common judge question in health-AI submissions, ‘how do you know this recommendation is trustworthy?’
- Close on the enterprise roadmap (Section 15) to show the judges this is not just a hackathon artifact — it has a credible path to real deployment.

6. System Architecture

The architecture is a four-layer federated system. Data and control mostly flow upward (PHC → District → National → BRICS), while models and recommendations flow back down. The design goal is that each layer can function even if the layer above it is temporarily unreachable — a PHC keeps working on local storage if it loses connectivity, and a district keeps working if the national layer is briefly down.

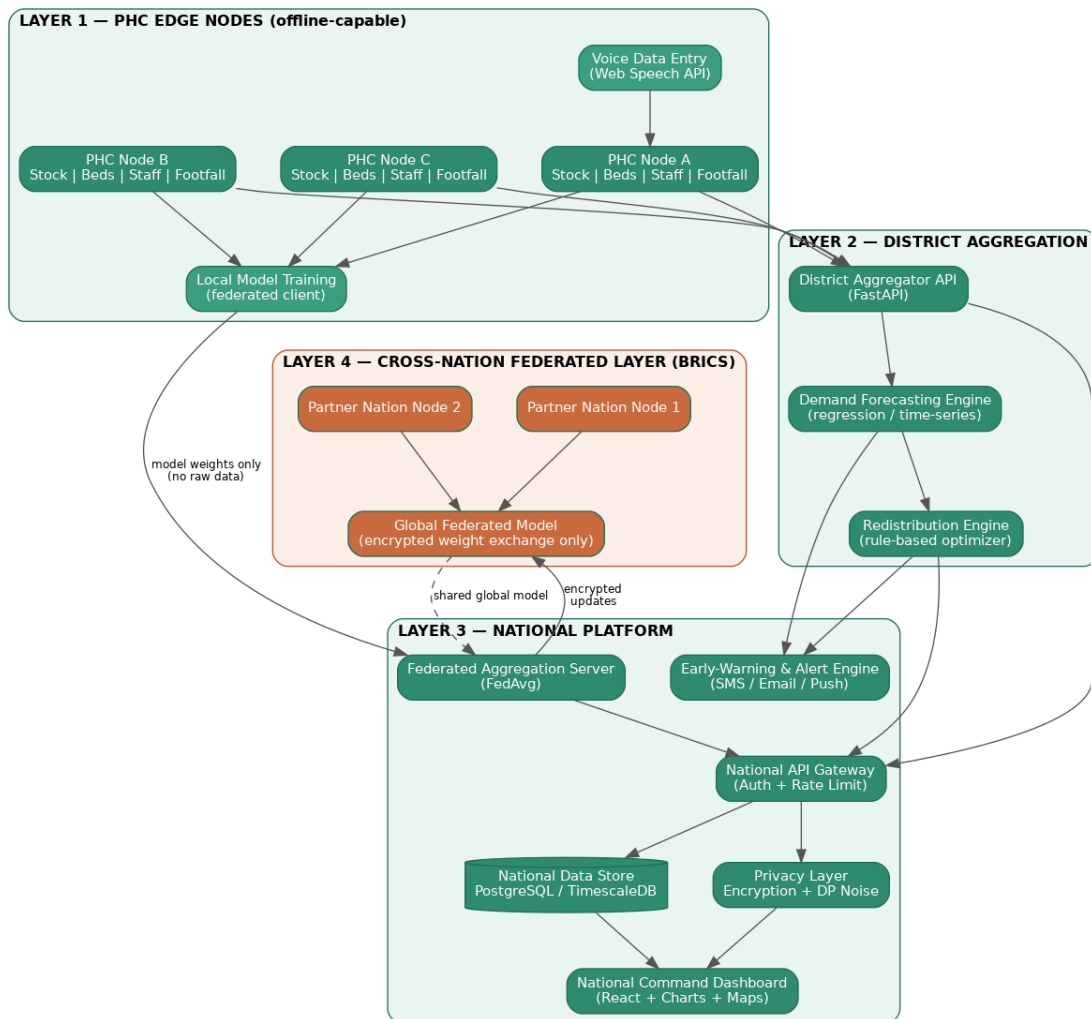


Figure 1 — Meridian four-layer architecture

6.1 Layer-by-layer breakdown

Layer 1 — PHC Edge Nodes

- Captures medicine stock, bed occupancy, staff attendance, and patient footfall.
- Runs on local storage (SQLite) so it keeps working without connectivity.
- Includes the voice-based data entry option for low-literacy or time-constrained staff.

- Trains a small local model (federated learning client) on its own data — this data never leaves the node in raw form.

Layer 2 — District Aggregation

- A district API aggregates data from every PHC in its district.
- Runs the demand forecasting engine per PHC and per medicine.
- Runs the redistribution engine across PHCs within the district.
- Surfaces a district heatmap: PHCs flagged by stock-out risk.

Layer 3 — National Platform

- National API gateway handles authentication and rate limiting for all district feeds.
- National data store holds aggregated (not raw patient-level) data for reporting and analytics.
- Federated aggregation server runs FedAvg (federated averaging) across the local models submitted by nodes, producing one improved global model.
- Privacy layer applies field-level encryption/tokenization and differential-privacy noise before any aggregate is displayed above the PHC layer.
- National command dashboard gives administrators the full picture: stock, beds, staff, forecasts, active redistribution recommendations, and the federated model health panel.
- Early-warning and alert engine pushes notifications to the right district/national officers when a risk threshold is crossed.

Layer 4 — BRICS Federated Layer

- Partner-nation nodes participate in the same federated averaging protocol as districts do nationally — only encrypted model weights cross the border, never raw records.
- For the hackathon build, this layer is a labelled simulation: 2–3 of your own PHC nodes are relabelled as partner nations to demonstrate the mechanism honestly, with the pitch stating plainly that live cross-border infrastructure is out of scope for a hackathon.
- The architecture is built so this layer can later connect to genuinely separate national deployments without a redesign — see Section 15.

6.2 Data sensitivity boundary

Key architectural principle

Raw, identifiable data (patient names, individual staff records) never crosses a network boundary. Only aggregate counts (with differential-privacy noise applied) and model parameters (weights, not training data) travel between layers. This is what makes the “shared predictive modelling across BRICS nations” requirement possible without violating any nation's health-data sovereignty.

7. Data & Decision Workflow

This is the end-to-end path a piece of data takes, from a PHC staffer logging a stock update to a national administrator seeing an actionable, explainable recommendation.

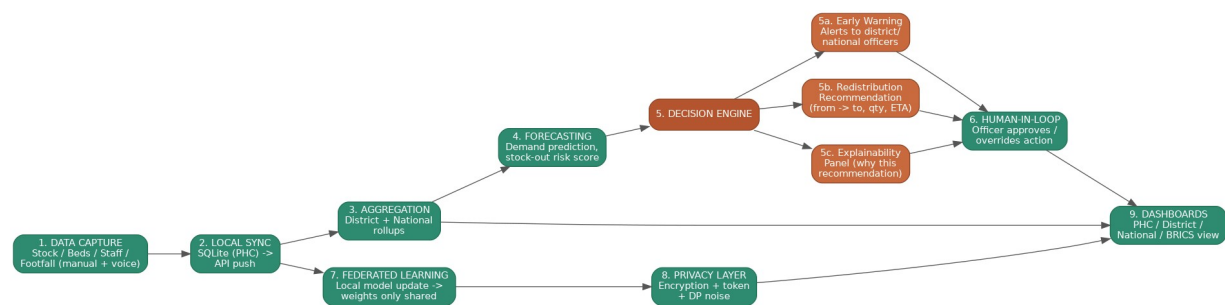


Figure 2 — End-to-end data and decision workflow

Step	What happens
1. Data capture	PHC staff log stock, bed, staff, and footfall data — typed or spoken (voice entry).
2. Local sync	Data is written to the PHC's local SQLite store, then pushed to the district API when connectivity allows.
3. Aggregation	District and national layers roll up PHC-level data into district and national views.
4. Forecasting	The forecasting engine computes predicted demand and a stock-out risk score per medicine per PHC.
5. Decision engine	Three things happen off the forecast: (a) early-warning alerts fire if risk crosses a threshold, (b) the redistribution engine proposes a transfer if a shortage/surplus pair exists, (c) the explainability panel attaches the underlying numbers to both.
6. Human-in-the-loop	A district or national officer reviews the recommendation and approves, adjusts, or overrides it — the system recommends, it does not act unilaterally.
7. Federated learning	In parallel, each PHC node trains its local model on its own data and shares only the resulting weights, never raw records.
8. Privacy layer	Weights and aggregate counts pass through encryption/tokenization and differential-privacy noise before being stored or displayed above PHC level.
9. Dashboards	PHC, district, national, and BRICS-simulation views all render from the same underlying pipeline, at different levels of aggregation.

Why human-in-the-loop matters for the pitch

Full autonomy over medicine redistribution is a hard sell to any real health administrator, and to most judges. Framing the system as a recommendation engine with an approval step — not an autonomous actuator — is both more realistic to build and more credible in front of judges who will ask “what happens if the AI is wrong?”

8. Technology Stack

The stack is chosen for two constraints at once: it has to be realistically buildable by a small team in a week, and every choice has a clear upgrade path to a production-grade equivalent, so the hackathon build is not throwaway code.

Layer	Hackathon-stage choice	Enterprise upgrade path	Why
Frontend	HTML/CSS/JS + Chart.js (or React if the team already knows it)	React/Next.js + a component library, native mobile app (React Native/Flutter)	A working simple dashboard beats a half-built fancy one under time pressure
Backend / API	FastAPI (Python)	FastAPI or Node/NestJS behind an API gateway, containerized	Fast to write, auto-generates a test UI at /docs, easy for forecasting code to live alongside
Database	SQLite (per PHC) syncing to a central SQLite/Postgres file	PostgreSQL + TimescaleDB for time-series consumption data, read replicas per region	Zero server setup during the hackathon; same SQL carries forward
Forecasting	NumPy linear regression / moving average on 7–14 day consumption windows	Proper time-series models (Prophet, ARIMA, or gradient-boosted models), retrained on a schedule	Explainable and fast to build; judges trust a model they can see the math of
Redistribution engine	Rule-based optimizer (Python)	Constraint/linear optimization (e.g. OR-Tools) factoring in transport cost, cold-chain constraints, multi-medicine trade-offs	Explainable, defensible under judge questioning, no training data required
Federated learning	Custom 3-node FedAvg demo in NumPy (local train + weight averaging script)	Flower or TensorFlow Federated with secure aggregation, differential-privacy budgets via Opacus/TF-Privacy	Demonstrates the real algorithm at small scale without a distributed-systems build-out
Privacy / security	Python cryptography (Fernet) field encryption + tokenization; random-noise aggregate counts	Formal differential privacy (epsilon/delta budget), HSM-backed key management, TLS everywhere, audit logging	Real, working privacy primitives that are honest about their limits
Alerting	In-dashboard alert panel; optional email via SMTP	SMS (Twilio/regional gateway), push notifications, escalation policies (PagerDuty-style)	In-app alerts are enough to demo the concept convincingly
Voice input	Browser Web Speech API (free, no external service)	Dedicated speech-to-text service with regional language support (e.g. Bhashini for Indian languages)	Zero cost, zero setup, works in any modern browser

Layer	Hackathon-stage choice	Enterprise upgrade path	Why
Infrastructure	Localhost for the demo; GitHub for source control	Kubernetes, per-nation sovereign cloud regions, CI/CD, infrastructure-as-code	Avoids losing hackathon time to deployment/config issues unrelated to the idea
Auth	None or a simple hardcoded demo login	OAuth2/SSO, role-based access control (PHC staff vs district vs national admin)	Not a judged capability at hackathon stage; real requirement for production

9. Twelve-Month Build Roadmap

Meridian is scoped as a one-year platform build, not a weekend project. The hackathon submission is Month 1, Week 1 — a working proof of concept that proves the architecture, not the finished product. Laying out the following eleven months up front is deliberate: it signals to judges, pilot partners, and any future collaborator that this is a team that has already thought past the demo, with a concrete, gated path from prototype to a platform a state health department could actually run. Each quarter below has a theme, a month-by-month breakdown, and an exit gate — measurable criteria that must be true before the next quarter starts.

Roadmap at a glance

Quarter	Theme	Headline milestone	Team size (by end of quarter)
Q1 — Months 1–3	Foundation & MVP	Working prototype + first pilot district running in shadow mode	2–3
Q2 — Months 4–6	Pilot & Validation	Published pilot accuracy report; second district onboarded	4
Q3 — Months 7–9	Scale & Harden	Multi-district rollout on production infrastructure; real federated learning framework live	6
Q4 — Months 10–12	Enterprise Readiness & Federation	Security audit passed; first real cross-institution federated pilot; Year-1 impact report	7–8

Q1 — Months 1–3: Foundation & MVP

Month 1 — Build the prototype

Week 1 is the hackathon sprint: seven days, one working proof of concept covering every pillar of the problem statement. Weeks 2–4 turn that prototype into something stable enough to put in front of a real pilot partner.

Week 1 — Sprint build

Day	Focus	Key deliverable
1	Setup, wireframes, architecture	Repo, stack installed, 3 dashboard wireframes, architecture doc drafted
2	Database + backend	SQLite schema, FastAPI CRUD endpoints, tested via /docs
3	PHC dashboard (real data loop)	One working screen proving input → database → dashboard end to end
4	District view + forecasting	District aggregation endpoint, regression-based demand forecast

Day	Focus	Key deliverable
5	Redistribution engine + explainability	Rule-based transfer recommendations with the numbers behind each one shown
6	Privacy + federated learning demo	Field encryption, DP-style noise, 3-node FedAvg demo
7	National dashboard + demo polish	Full national view, BRICS-simulation panel, rehearsed demo, submission packaged

Weeks 2–4 — Harden the prototype

Week	Focus	Key deliverable
2	Test coverage + bug fixing	Unit tests on the forecasting, redistribution, and federated-averaging functions; fix issues found post-demo
3	Containerization + documentation	Dockerized backend, a README a stranger could follow, an updated architecture doc reflecting what actually got built
4	Pilot outreach begins	Shortlist of 3–5 candidate district health departments or NGO health programs; first outreach emails sent

Month 2 — Pilot partner engagement

- Formalize outreach into at least one conversation with a district health department, PHC network operator, or public-health NGO willing to pilot in shadow mode (the platform runs and makes recommendations, but no real workflow depends on it yet).
- Draft and sign a lightweight data-sharing / pilot agreement — no PII changes hands beyond what the pilot partner already collects internally.
- Rebuild the data model against the pilot partner's real PHC structure (facility count, medicine formulary, reporting cadence) instead of the synthetic hackathon dataset.

Month 3 — Shadow-mode pilot goes live

- Deploy Meridian in shadow mode across the pilot district's PHCs — real data flows in, forecasts and redistribution recommendations are generated, but nothing is auto-approved.
- Collect a baseline of forecast accuracy against what actually happened, and log every redistribution recommendation for later review.
- Run structured feedback sessions with PHC staff on the data-entry flow, including the voice-input feature, and fix the friction points that surface.

Q1 exit gate

Working prototype demoed publicly (hackathon submission) • shadow-mode pilot live in at least 1 district for 2+ weeks • forecast backtest error within an agreed tolerance • PHC staff able to complete a full data-entry cycle without assistance.

Q2 — Months 4–6: Pilot & Validation

Month 4

- Onboard a second, structurally different pilot district (different geography or facility density) to stress-test assumptions baked in from the first.
- Begin the move from noise-based aggregate privacy to a formally scoped differential-privacy budget.
- First hire: a backend/ML engineer to own forecasting and redistribution model quality full-time.

Month 5

- Upgrade the redistribution engine from pure rule-based matching to constraint-based optimization (transport cost, cold-chain limits) using real distance and route data for the pilot regions.
- Make the dashboard mobile-responsive — most PHC-level users will be on a phone, not a desktop.

Month 6

- Publish a short, honest pilot accuracy and impact report — forecast error, stock-outs prevented, redistribution recommendations acted on — the evidence base for every future conversation.
- Open conversations with a state health department and at least one funding path (public-health NGO, multilateral digital-health initiative, or state GovTech budget line).

Q2 exit gate

2 districts live · published pilot report with real accuracy numbers · redistribution engine running on real geographic data · at least one funding or state-level conversation underway.

Q3 — Months 7–9: Scale & Harden

Month 7

- Migrate from SQLite to PostgreSQL + TimescaleDB and stand up a CI/CD pipeline — the platform stops depending on any one laptop.
- Begin multi-district rollout: 5–10 PHCs live simultaneously.
- Hire: a mobile engineer and a DevOps/infrastructure engineer.

Month 8

- Replace the hackathon-stage NumPy federated-averaging demo with a real framework (Flower or TensorFlow Federated) supporting secure aggregation.
- Add formal authentication and role-based access control distinguishing PHC staff, district officers, and national administrators.

Month 9

- Pilot HL7 FHIR interoperability with one existing state or national health information system, so Meridian augments rather than replaces what health workers already use.

- Ship a mobile app beta (React Native) for PHC staff, offline-first.
- Open first-contact conversations with a counterpart team or health ministry in one other BRICS nation about a genuine federated pilot.

Q3 exit gate

5–10 PHCs live on production infrastructure · federated learning running on a real framework with secure aggregation · mobile beta in the hands of real PHC staff · one active cross-nation conversation in progress.

Q4 — Months 10–12: Enterprise Readiness & Federation

Month 10

- Commission an independent security audit and a compliance review against India's DPDP Act (and the equivalent framework in any partner nation engaged so far).
- Move production infrastructure to Kubernetes with per-region deployment support.
- Hire: a security/compliance lead and a partnerships/government-relations lead.

Month 11

- Run the first real cross-institution federated pilot — a genuinely separate partner deployment (a different state, or, if the Month 9 conversation matured, a different nation) exchanging only encrypted model weights with the national server.
- Harden the national command dashboard based on a full quarter of real multi-district usage data.

Month 12

- Publish a Year-1 impact report: stock-outs prevented, districts covered, forecast accuracy trend, and what the federated layer improved that isolated per-district models could not.
- Take the Year-1 report into a formal state or national rollout proposal, and, if appropriate, a funding conversation sized for national scale rather than a pilot.

Q4 exit gate

Security audit passed with no critical findings · at least one genuine cross-institution federated pilot running outside the founding team's own infrastructure · Year-1 impact report published · a live rollout or funding conversation at state/national scale.

Why show a 12-month roadmap in a hackathon submission

Most hackathon teams present a weekend's worth of work as if it's the finished idea. Showing the next eleven months — with concrete, measurable exit gates per quarter — tells judges and any future partner that the team understands what it actually takes to run this in a real health system, and has already planned for it. It is the difference between “we built a demo” and “this is month one of something we intend to run.”

10. Team & Execution Plan

The founding build (Month 1) needs to move fast with a small team. Every hire after that is tied to a specific quarter in Section 9, not added speculatively — each new role exists because a specific milestone in the roadmap needs it.

10.1 Month 1 — Founding build

Track	Owns	Primary tech
Track A — Backend & AI	Database, API endpoints, forecasting model, redistribution engine, encryption, federated learning scripts	Python, FastAPI, SQLite, NumPy, cryptography
Track B — Frontend & UX	All three dashboards, forms, charts, voice input wiring, explainability panel UI, demo polish	HTML/CSS/JS or React, Chart.js, Web Speech API

If a third person is available in Month 1, add a dedicated validation/docs/pitch track:

Track	Owns
Track C — Validation, Docs & Pitch	Runs the validation checklist (Section 11) daily against whatever A and B shipped that day, maintains the README and architecture doc, builds the slide deck incrementally instead of scrambling on Day 7, and owns rehearsing the demo script

Solo builder note: follow the phases in Section 9's Week 1 table sequentially, exactly as ordered — the order is chosen so each day's output is demoable on its own, so a solo builder who runs out of time still has a complete, working story (visibility → forecast → redistribution) rather than several half-finished layers.

10.2 Hiring plan — Year 1

When	Role added	Why this role, at this point
Month 1	2 co-founders (Backend/AI + Frontend/UX)	Builds and demos the prototype; no hires needed yet
Month 4	Backend/ML engineer	Q2 needs someone owning forecasting and redistribution model quality full-time while the co-founders manage the pilot relationship
Month 7	Mobile engineer	Q3's multi-district rollout depends on the mobile beta most PHC staff will actually use
Month 7	DevOps/infrastructure engineer	Migrating off SQLite/localhost onto real production infrastructure with CI/CD is a full-time job, not a side task
Month 10	Security/compliance lead	Q4's security audit and DPDP-equivalent compliance review need a dedicated owner, not a part-time effort bolted onto engineering
Month 10	Partnerships/government-relations lead	Turning pilot results into a state or national rollout conversation is a distinct skill set from building the platform

10.3 Advisory needs

Bring these on informally (equity or advisory-fee basis) starting Q2, ahead of when the team formally needs them full-time:

- A public-health domain advisor — someone who has actually run or overseen a PHC network, to sanity-check every assumption baked into the data model and redistribution logic.
- A data-privacy/legal advisor — familiar with India's DPDP Act and, once cross-nation conversations start in Q3, the equivalent frameworks in partner nations.
- A government-relations advisor — for the Q4 state/national rollout conversation, ideally someone who has navigated a GovTech procurement process before.

11. Validation Plan — Prove It Actually Works

A recommendation engine that looks right in a screenshot but hasn't been checked against known inputs is a liability — in front of hackathon judges, in front of a pilot health officer, and in front of an investor doing diligence. The checklist below is the standing validation practice for the platform, not a one-time pre-demo task; it should run continuously as the roadmap in Section 9 progresses.

11.1 Technical validation checklist

Layer	What to validate	How
API layer	Every endpoint returns correct data for known inputs	Use FastAPI's auto-generated /docs page to call each endpoint directly with test values before wiring the frontend
Data loop	A change made in the UI is reflected correctly and promptly in the database and back in the dashboard	Manual test: update a stock value in the form, confirm the number changes on screen and in the DB file
Forecasting model	Predictions are directionally correct and the error is small enough to trust in a pitch or a pilot report	Backtest: hide the last several days of the usage time-series, predict them from the days before, compare predicted vs. actual (report the error, e.g. mean absolute error)
Redistribution engine	The engine recommends the correct transfer for a known shortage/surplus scenario	Construct a test case with a known-correct answer (PHC-A short by 40 units, PHC-B surplus of 100 units, 20 minutes apart) and confirm the engine's output matches
Federated learning	The “global model” is genuinely the average of the local models, and raw data never left its node	Print each local model's weights and the averaged result, confirm the arithmetic by hand or with an assert statement; separately, confirm by code review that only weight arrays (not dataframes) are passed to the aggregation function
Privacy layer	Encrypted fields are unreadable without the key; noised aggregates are close to, but not identical to, the true count	Try reading the raw database file and confirm the patient/staff identifier field is ciphertext, not plaintext; compare a few noised vs. true counts to confirm the noise bound
End-to-end demo	The full 3-minute script (Section 13) runs cleanly, twice, on the presentation machine and network	Full dry run, twice, including a spoken run-through with a timer; keep a backup screen recording in case of live failure

11.2 Quarterly exit-gate criteria

The same checklist above, rolled up into what must be true before the roadmap moves to the next quarter — see Section 9 for full context on each gate.

Quarter	Exit criteria
Q1	Working prototype demoed publicly · shadow-mode pilot live 2+ weeks in 1 district · forecast backtest error within tolerance · PHC staff can complete a data-entry cycle unassisted

Quarter	Exit criteria
Q2	2 districts live · published pilot accuracy report · redistribution engine using real geographic data · a funding or state-level conversation underway
Q3	5–10 PHCs live on production infrastructure · federated learning on a real framework with secure aggregation · mobile beta in real use · one active cross-nation conversation
Q4	Security audit passed, no critical findings · a genuine cross-institution federated pilot live · Year-1 impact report published · a live rollout or funding conversation at scale

11.3 Anticipated questions — prepare answers in advance

1. “How do you know this recommendation is trustworthy?” → point to the explainability panel and the validation numbers above.
2. “Is this really federated learning, or just an API call?” → show the three local-training scripts and the averaging script, and state clearly that raw data never left each node.
3. “What happens if the AI is wrong?” → point to the human-in-the-loop approval step (Section 7, Step 6) — the system recommends, an officer decides.
4. “Is the BRICS layer real cross-border infrastructure?” → be direct that it's a labelled simulation at hackathon stage, and point to Section 9's Q3/Q4 milestones for exactly when and how it becomes real.
5. “Why should we believe you'll actually execute a 12-month plan off the back of a hackathon build?” → point to Section 9's quarterly exit gates and Section 10's hiring plan — each is a measurable commitment, not an aspiration, and Month 1 itself is the first proof of execution speed.

12. Impact

The figures below are illustrative, framed for a pitch narrative — they should be presented to judges as reasoned estimates based on how the mechanism works, not as measured outcomes, since the system has not yet run in a live health system. Being explicit about that distinction is itself part of what judges are listening for.

Dimension	Mechanism	Illustrative impact framing
Stock-out reduction	Early warning fires before stock hits zero, giving time to redistribute or reorder	Facilities with an active early-warning + redistribution loop should see materially fewer complete stock-outs than facilities relying on manual, periodic stock checks
Emergency response time	Real-time visibility replaces phone/paper reporting chains that can take days to surface a shortage	A predicted shortage is visible to a district officer in near real time instead of after the fact
Resource utilization	Redistribution moves surplus stock to where it's needed instead of letting it expire unused at a low-demand PHC	Reduces both stock-outs at one facility and wastage at another from the same root cause: uneven visibility
Equity across PHCs	National dashboard surfaces under-resourced PHCs that would otherwise be invisible to state-level planners	Smaller, remote PHCs get the same visibility as large urban ones — the platform doesn't require a facility to self-report loudly to be noticed
Data sovereignty for BRICS cooperation	Federated learning improves shared forecasting accuracy without any nation transmitting raw patient data across borders	Makes cross-border predictive cooperation politically and legally feasible in a way that centralized data-sharing is not

12.1 Alignment with existing policy frameworks

- UN Sustainable Development Goal 3 (Good Health and Well-Being) — directly targets health system resilience and access to essential medicines.
- UN Sustainable Development Goal 17 (Partnerships for the Goals) — the BRICS federated layer is a concrete technical instance of cross-border health cooperation.
- India's Ayushman Bharat Digital Mission (ABDM) and equivalent national digital health stacks in other BRICS nations — Meridian is designed to eventually interoperate with these via standard health data exchange formats (see Section 15), not replace them.

13. Demo Script (3 Minutes)

A single, continuous scenario is far more convincing than jumping between disconnected feature demos. Use one crisis narrative and let each screen answer the next question the audience would naturally have.

6. A disease outbreak begins in the demo region — trigger a spike in footfall and medicine usage at PHC-A manually, using the update form (or the voice input, for extra effect).
7. Switch to the forecast panel: it shows a predicted stock-out in roughly 36 hours, with the explainability numbers visible underneath.
8. Switch to the district view: the redistribution engine recommends a transfer from PHC-B (which has surplus) to PHC-A, with quantity and ETA shown.
9. Show the federated learning panel: each PHC-node's local model, and the averaged global model, with the line stated plainly — “no raw patient data ever left its own PHC.”
10. Close on the national dashboard showing the crisis resolved, and briefly show the BRICS-simulation panel, stating clearly that it is a labelled simulation of the same mechanism at cross-border scale.

Closing line

End with one sentence that ties the whole demo back to the brief: “This system gives national-scale visibility and response — while every patient record stays exactly where it was created.”

14. Submission Checklist

- A working local demo (backend + frontend + AI scripts) in a GitHub repo, with a README that documents setup steps clearly enough for a judge to run it themselves if asked.
- The architecture diagram (Figure 1 in this document, or your own version of the same box diagram).
- A slide deck: problem → solution → architecture → live demo → what's simplified and why → impact.
- A short written submission (5–10 pages is enough for a working prototype — judges weigh the deck and the live demo far more heavily than document length).
- A backup screen recording of the full demo, in case of live technical issues on presentation day.

15. From Hackathon Prototype to Enterprise Product

Section 9 laid out when each quarter's work happens; this section lays out what changes, feature by feature. Every simplification made for the Month 1 build (Section 8) has a named, credible upgrade path, each tied to a specific quarter above — this section is what turns “we won a hackathon” into “we have a product a health ministry or a multilateral health body would actually evaluate.”

Hackathon-stage simplification	Enterprise-grade evolution
Custom NumPy FedAvg demo across 3 local scripts	Production federated learning framework (Flower or TensorFlow Federated) with secure aggregation protocols, so no single party — including the national server — can see any individual node's raw model update
Random-noise aggregate privacy	Formal differential privacy with a defined epsilon/delta budget, implemented via established libraries (Opacus, TensorFlow Privacy), independently auditable
Fernet field-level encryption	Full key-management via a hardware security module or managed KMS, TLS in transit everywhere, per-nation encryption-at-rest key ownership for data sovereignty
SQLite per PHC	PostgreSQL + TimescaleDB for time-series data, regional read replicas, per-nation sovereign cloud hosting where required by law
Rule-based redistribution engine	Constraint/linear optimization (e.g. Google OR-Tools) accounting for transport cost, cold-chain requirements, multi-medicine trade-offs, and multi-hop transfers
Linear regression forecasting	Proper time-series forecasting (Prophet, ARIMA, or gradient-boosted trees), retrained on a schedule, monitored for drift
Simulated BRICS nodes	Genuine multi-nation deployment: each partner nation runs its own instance of the platform, federating only at the model-weight level via a formal cross-border data-sharing agreement
No formal auth	OAuth2/SSO with role-based access control distinguishing PHC staff, district officers, and national administrators
Manual demo alerts	SMS via a regional gateway, push notifications, and an escalation policy so an unacknowledged critical alert automatically reaches the next tier
No mobile app	A React Native or Flutter offline-first mobile app for PHC staff, syncing opportunistically when connectivity returns

15.1 Additional enterprise-only features (deliberately cut from the hackathon scope)

- Digital medicine passport — batch IDs and supply-chain provenance tracking to detect counterfeit or diverted medicine, potentially blockchain-anchored for tamper-evidence.

- IoT cold-chain sensors — real-time temperature monitoring for vaccine and temperature-sensitive medicine storage, feeding directly into the same stock-visibility pipeline.
- EHR/HMIS interoperability — integration via HL7 FHIR standards so the platform augments existing national health information systems (like India's ABDM) rather than requiring PHCs to abandon what they already use.
- Emergency simulation engine — a proper scenario-planning tool for health administrators to stress-test the redistribution network against hypothetical outbreak patterns, beyond the scripted demo version used for the hackathon pitch.

15.2 Path to a real pilot

11. Identify 1–2 district health departments willing to run a pilot on real (or shadow/parallel) data without replacing their existing reporting process.
12. Run the platform in shadow mode for 4–8 weeks: it makes recommendations, but no workflow depends on it yet, so forecast accuracy and redistribution quality can be measured against what actually happened.
13. Use that pilot data to publish a short accuracy/impact report — this becomes the evidence base for a wider state or national rollout conversation.
14. Approach funding through GovTech SaaS licensing to state health departments, or multilateral health cooperation funding (e.g. WHO digital health initiatives, BRICS health cooperation mechanisms) for the cross-border federated layer specifically.

16. Risks & Mitigations

Risk	Mitigation
Running out of time and having a broken demo in Week 1	Follow the Week 1 sprint table in Section 9 exactly — it's ordered so every earlier day is independently demoable if later days don't get finished
Judges doubting the federated learning is real	Show the actual local-training and averaging scripts running live, not just a panel with numbers on it
Over-claiming the BRICS cross-border layer	State plainly and early in the pitch that it is a labelled simulation, and explain exactly what would make it real (Section 15)
Forecast or redistribution engine giving a wrong-looking answer live	Run the validation checklist (Section 11) against your actual demo dataset before presentation day, not just against toy examples
Live demo failure (network, laptop, projector)	Keep a backup screen recording of a full successful run, and rehearse an offline-capable fallback path if possible
Scope creep — building the digital medicine passport or emergency simulation engine instead of finishing the core loop	These are explicitly enterprise-roadmap items (Section 15.1), not hackathon-scope — cut them if time is tight
No pilot district signs on in Month 2, stalling the Q1 exit gate	Start outreach to a shortlist of 3–5 candidates in Week 1, in parallel with the build, rather than waiting until the demo is finished — see Section 9, Month 1
Roadmap slips because the founding team can't hire fast enough	The hiring plan (Section 10.2) is tied to milestones, not calendar dates — a slipped hire delays the next quarter's start rather than breaking the whole plan; use the advisory layer (Section 10.3) to cover gaps in the meantime
A reviewer assumes the 12-month plan is aspirational slideware	Anchor every quarter to its exit gate (Section 9) and cross-reference the validation plan (Section 11.2) — the roadmap is written as a series of falsifiable claims, not a vision statement

17. Appendix

17.1 Suggested repository structure

meridian/ backend/ (FastAPI app, database) frontend/ (dashboards) ai/ (forecasting + federated learning scripts) docs/ (architecture diagram, this document) README.md

17.2 Core database schema

Table	Key fields
medicine_inventory	phc_id, medicine_name, quantity, updated_at
bed_status	phc_id, total_beds, occupied_beds, updated_at
staff_attendance	phc_id, staff_id, present, date
patient_footfall	phc_id, date, count

17.3 Minimal forecasting function

```
import numpy as np

def forecast_next_day(daily_usage):
    x = np.arange(len(daily_usage))
    slope, intercept = np.polyfit(x, daily_usage, 1)
    predicted = slope * len(daily_usage) + intercept
    return {"predicted_demand": max(0, round(predicted))}
```

17.4 Minimal federated averaging function

```
import numpy as np

def federated_average(weight_list):
    return np.mean(weight_list, axis=0)
```

17.5 Reference

Hackathon: hack2skill — Build with AI: Code for Communities (2nd Edition). This document was prepared as a technical handoff for the engineering build of the Meridian submission.